

1.2 - DEEP NEURAL NETWORKS

CHAPTER INDEX

- ☐ [DEEP NEURAL NETWORKS](#)
 - [WHY WE NEED "DEEP" NETWORKS](#)
 - [EXPLODING/VANISHING GRADIENT PROBLEM](#)
 - [PARAMETER INITIALISATION](#)
 - [BATCH NORMALISATION](#)
 - [FUNCTION CLASSES](#)
- ☐ [ACTIVATION FUNCTIONS](#)
 - [ReLU](#)
 - [LEAKY ReLU, ELU](#)
 - [SReLU](#)
 - [APLU](#)
 - [MEXICAN ReLU](#)

DEEP NEURAL NETWORKS

In > [1.0 - NEURAL NETWORKS INTRODUCTION](#) we introduced neural networks, the **perceptron** and the **multi-layer perceptron**, but a lot of the modern-day **hype** regarding AI and neural networks revolves around **deep neural networks**:

DEEP NEURAL NETWORK (DNN)

Neural network with **at least two hidden layers**.

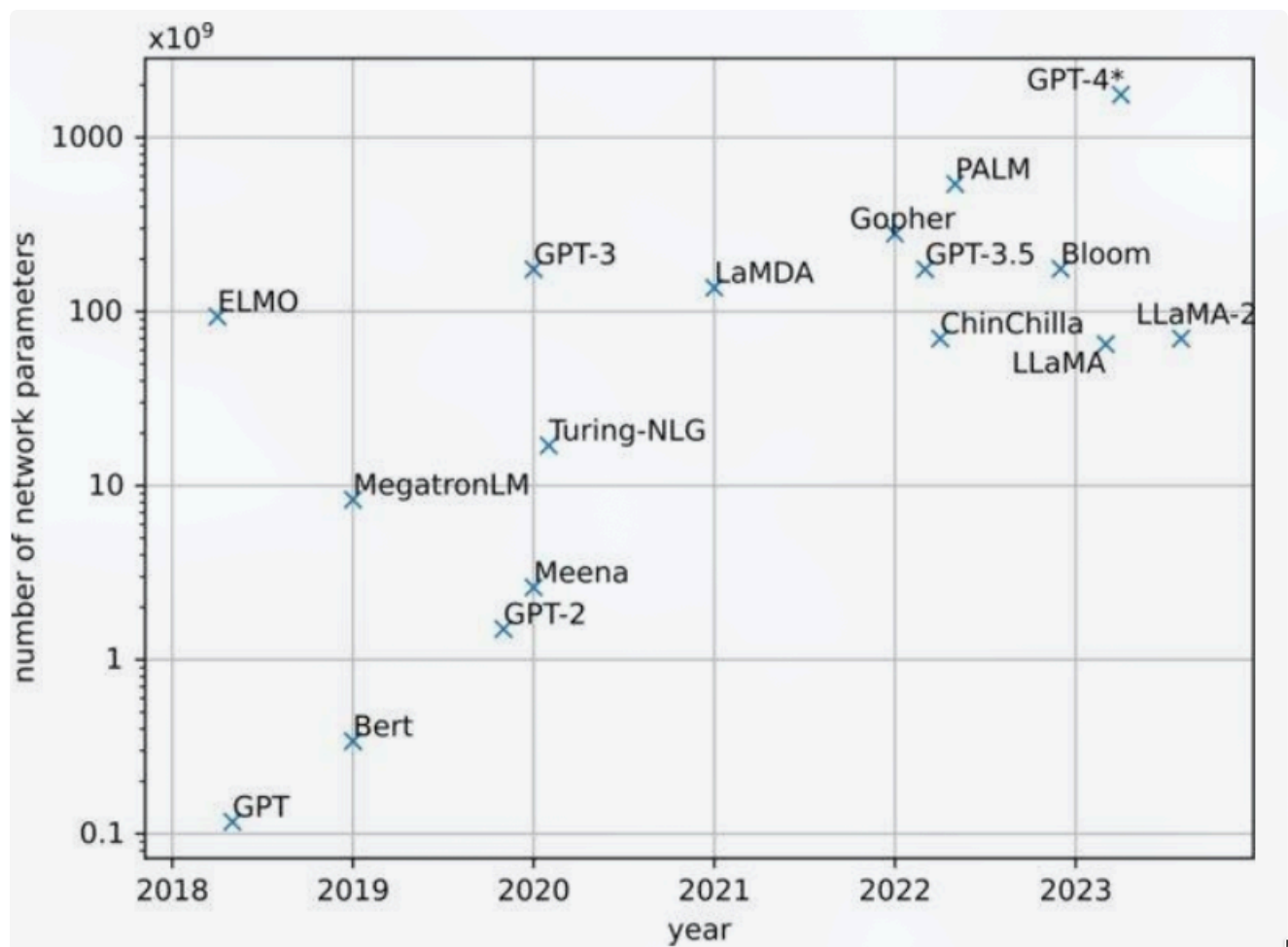
Thus, DNNs are, to put it in *extremely simple terms*, a **big, scaled-up version of the MLP**.

The idea behind DNNs was **partially inspired by human reactions to stimuli**: there are *about ten levels between the retina and actuator muscles*.

Though in practice, as time went on, the **number of layers** in DNNs **increased** way past this threshold: the most common modern models consist of **7 to ~150 layers**.

Notice that **depth** is **only one of the complexity factors** of a DNN.

The **total number of neurons**, the **amount of connections and weights** also play an important role.



🔍 OBSERVATIONS

- The **greater the number of weights**, the **more complex training** is.
- A **large number of neurons** makes **propagation** (both forward and backwards) **more computationally expensive**.

Notice that, in the examples presented in > [1.0 - NEURAL NETWORKS INTRODUCTION](#), the neurons had *all possible connections with adjacent layers*; networks of this kind are called **fully connected DNNs (FCDNN)**.

This "full connection" is **not a requirement**: various kinds of DNNs are **not fully connected**.

An example is given by **Convolutional Neural Networks**, which are discussed in > [2.0 - CONVOLUTIONAL NEURAL NETWORKS](#).

WHY WE NEED "DEEP" NETWORKS

We've discussed the **universal approximation theorem** in > [1.0 - NEURAL NETWORKS INTRODUCTION > UAT AND MULTILAYER NETWORKS](#), which proves that *MLPs with one hidden layer are universal approximators*.

Notice that this is an **existence theorem**, meaning it **doesn't tell** us anything about **how to actually build** (that is, how many neurons we need) these **approximators**.

This is one of the **main reasons** why, after a period of *initial excitement* for 3-layer MLPs, **attention shifted to deep neural networks**: once **training algorithms were developed** and **hardware became more powerful** (see > [1.0 - NEURAL NETWORKS INTRODUCTION > BACKPROPAGATION](#)), it became **much easier** to **build DNNs** than to find the right MLP model predicted by the UAT.

CURIOUS FACT

In 1960 the idea of the "**grandmother cell**" was proposed: it was thought to be a *neuron that represents a complex and specific concept that fires when this concept is encountered*.

The idea was **quickly proven false** by psychologists, but an experiment found it to be **true for DL models**.

In 2012, a research team led by Andrew Ng and Jeff Dean fed **10 million random YouTube videos** to the **Google Brain Simulator**. It was then found that the model had developed a **peculiar neuron** which fired **only for videos containing cats**.

EXPLODING/VANISHING GRADIENT PROBLEM

One of the main challenges that needed to be tackled to make DNNs work was the **vanishing** (and also *exploding*) **gradient problem**.

Recall that the **backpropagation** algorithm relies on the following two equations:

$$\delta^{(L)} = \nabla_z J \odot f'(\text{net}^{(L)})$$

$$\delta^{(l)} = \left(w^{(l)} \delta^{(l+1)} \right) \odot f'(\text{net}^{(l)})$$

That is, at each step, we compute the **gradient with respect to one layer's weights** and then we **keep propagating it backwards for each layer** in the network.

This introduces **two possible problems**, because we may be facing **parameter updates** that are either:

- **Excessively large**, destroying our model (*exploding gradient problem*).
- **Excessively small**, making *learning impossible* as parameters barely move on each update (*vanishing gradient problem*).

Imagine we're **starting with a low value** of $\nabla_z J$: as a consequence, $\delta^{(L)}$ will **also be a small number** (unless $f'(\text{net}^{(L)})$ is somehow huge); but this will also make $\delta^{(L-1)}$ a small number, and so on.

That is, when reaching the final steps we obtain numbers that result from **long streaks of multiplications of small numbers**, thus making each $\delta^{(l)}$ **progressively smaller**.

The same reasoning applies to *excessively large numbers*.

💡 SOURCES OF THE PROBLEM

The problems can be solved in two ways, each acting on *one* of the causes:

1. Clever **parameter initialisation**: intelligently choosing the initial weights means $w^{(l)}$ can *offset problems introduced by* $f'(\text{net}^{(l)})$.
2. Different **choice of activation function**: the sigmoid's derivative is *practically zero* at the *two ends of its domain*, thus choosing an activation function with a *better derivative profile* can reduce the risk of facing vanishing gradients.

PARAMETER INITIALISATION

As seen above, one way of addressing (or at least **minimising**) the issues related to the calculation of the gradient for backpropagation is through a **careful initialisation** of the weights.

We know weights are **initialised at random** before training, thus the question is: **what kind of distribution should they follow?**

💡 IDEA

We can **look at the distributions of the outputs** of each layer (which will *depend on the weights*) and **impose** that **their variances remain fixed throughout the layers**.

Let's consider the **distribution** of an **output** o_i (of one of the *hidden neurons*) during **forward-propagation** for some **fully connected layer without non-linearities**.

With n_{in} inputs (neurons of the *previous layer*) x_j and their *associated weights* w_{ji} for the considered layer, an output is given by:

$$o_i = \sum_{j=1}^{n_{\text{in}}} w_{ji} x_j$$

The weights w_{ji} are all **drawn independently** from the **same distribution**.

Let us assume that this distribution has **zero mean** and **variance** σ^2 .

Furthermore, we also assume that the inputs x_j **also have zero mean** and **variance** γ^2 , and that they are **independent** of w_{ji} and **of each other**.

In this case, we can compute the *mean and variance* of o_i as follows:

$$\begin{aligned}
\mathbb{E}[o_i] &= \sum_{j=1}^{n_{in}} \mathbb{E}[w_{ji}x_j] \\
&= \sum_{j=1}^{n_{in}} \mathbb{E}[w_{ji}]\mathbb{E}[x_j] \\
&= 0
\end{aligned}$$

$$\begin{aligned}
\text{Var}[o_i] &= \mathbb{E}[o_i^2] - \mathbb{E}[o_i]^2 \\
&= \sum_{j=1}^{n_{in}} \mathbb{E}[w_{ji}^2 x_j^2] - 0 \\
&= \sum_{j=1}^{n_{in}} \mathbb{E}[w_{ji}^2] \mathbb{E}[x_j^2] \\
&= n_{in} \sigma^2 \gamma^2
\end{aligned}$$

One way to **keep the variance fixed** (signals don't vanish or explode) is to set:

$$n_{in} \sigma^2 = 1$$

Let us now **consider backpropagation**.

We face a similar problem, with the only difference being the fact that **propagation happens from the output to input layer**.

Thus, with the same reasoning we see that the **gradient's variance blows up unless** we set:

$$n_{out} \sigma^2 = 1$$

Where n_{out} is the *number of outputs of this layer* (number of neurons in the considered layer).

Clearly, this leaves us in a dilemma: **we cannot possibly satisfy both conditions simultaneously**.

Instead, we **simply try to satisfy**:

$$\frac{1}{2}(n_{in} + n_{out})\sigma^2 = 1$$

Which is **equivalent to**:

$$\sigma = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

XAVIER INITIALISATION

When **weights** are initialised by randomly drawing them from a **distribution**

with the above presented **variance**, we talk about **Xavier initialisation**.

Typically, Xavier initialisation samples weights from a *Gaussian distribution* with *zero mean* and *variance*:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Notice that this is just **one possible choice**: the distribution doesn't necessarily have to be Gaussian, it just needs to have the **correct mean and variance**.

Thus, we could also adapt Xavier's intuition to **choose the right variance** if we want to **sample weights from a uniform distribution**.

Note that the uniform distribution $U(-a, a)$ has variance $a^2/3$. Plugging this into our condition for σ^2 yields the **value of a** for the **correct interval**:

$$U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

CLARIFICATION

Remember that n_{in} and n_{out} are **different for each layer**, they are **not the network's number of input and output neurons**.

Notice that, though the assumption for nonexistence of nonlinearities in the above reasoning can easily be violated in neural networks, the proposed method turns out to **work well in practice**.

BATCH NORMALISATION

For a *deep neural network*, as we train, the **variables** (e.g. affine transformation outputs in an MLP) in **intermediate layers** may take values with **widely varying magnitudes**, both along the *layers from the input to the output* and *across units in the same layer*, as well as *over time* due to *our updates to the model parameters*.

It's been **informally postulated** that this drift in the distribution of such variables could **hamper the convergence** of the network.

Intuitively, one might conjecture that if one layer has variable values that are 100 times that of another layer, this might **necessitate compensatory adjustments in the learning rates**.

Batch normalisation is a form of *regularisation* that addresses this issue by **normalising the inputs in each training iteration** and applying a *scale coefficient* and a *scale offset*.

The normalisation relies on the mean and variance of the current minibatch, hence the name.

Formally, denoting $\mathbf{x} \in \mathcal{B}$ an input of the current batch, batch normalisation transforms $\mathbf{x}$ according to the following expression:

$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta$$

Where $\hat{\mu}_{\mathcal{B}}$ is the sample mean and $\hat{\sigma}_{\mathcal{B}}$ is the sample standard deviation of the minibatch $\mathcal{B}$; and γ, β are the **scale coefficient and offset** (respectively), with the *same shape* as $\mathbf{x}$.

Notice that γ and β need to be **learnt jointly with the other model parameters**.

Formally, the mean and variance are computed as follows:

$$\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}$$
$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon$$

Notice that we add a small constant ϵ to the variance estimate to *ensure we never attempt division by zero*.

Because these two parameters are **estimated on the minibatch** and **not the entire data set**, they **introduce noise**. One might think that this noisiness should be a problem, but as it turns out, this is **actually beneficial**.

This turns out to be a *recurring theme* in deep learning. For reasons that are **not yet well-characterised theoretically**, various sources of **noise** in optimisation lead to **faster training** and **less overfitting**.

ON THE UTILITY OF NOISE

Recall that we **do not** even **know why simpler DNNs generalise well** in the first place.

Even with **dropout**, **weight decay** and **regularisation techniques**, they remain **so flexible** that their ability to generalise to unseen data **cannot be explained** via conventional approaches to learning theory.

Going back to the actual application of batch normalisation, we've only vaguely talked about "**layer variables**", without specifying what we mean by that.

This is because batch normalisation can be **applied at different stages of computation** within layers:

WHERE TO APPLY BATCH NORMALISATION

When applying batch normalisation, the authors of the original paper performed it **after the affine transformations** and **before the activation**

function, but later applications sometimes *insert batch normalisation right after the activation function*.

🔗 BATCH NORMALISATION IN PREDICTION MODE

Once *training is complete*, why would we want the *same image* to be *classified differently depending on the batch* it happens to reside?

Thus, once the model is trained, we **calculate the means and variances** of each layer's variables based on the **entire data set**.

FUNCTION CLASSES

As we design increasingly deeper networks it becomes imperative to understand how **adding layers can increase** the complexity and **expressiveness of the network**.

Even more important is the ability to design networks where adding layers makes network **strictly more expressive** rather than *just different*.

This is where **function classes** come into play.

Consider $\mathcal{F}$, the **class of functions** that a specific network architecture **can reach**. That is, for all $f \in \mathcal{F}$ there **exists some set of parameters** (e.g. weights and biases) that can be **obtained** through **training** on a suitable dataset.

Let us assume that f^* is the "**truth**" **function** that we really would like to find. If it is in $\mathcal{F}$ we are in good shape, but typically we are not quite so lucky.

Instead, we will try to find some $f_{\mathcal{F}}^*$ which is our **best guess** within $\mathcal{F}$. For instance, given a dataset with features $\mathbf{X}$ and labels $\mathbf{y}$, we might try finding it by solving the following optimisation problem:

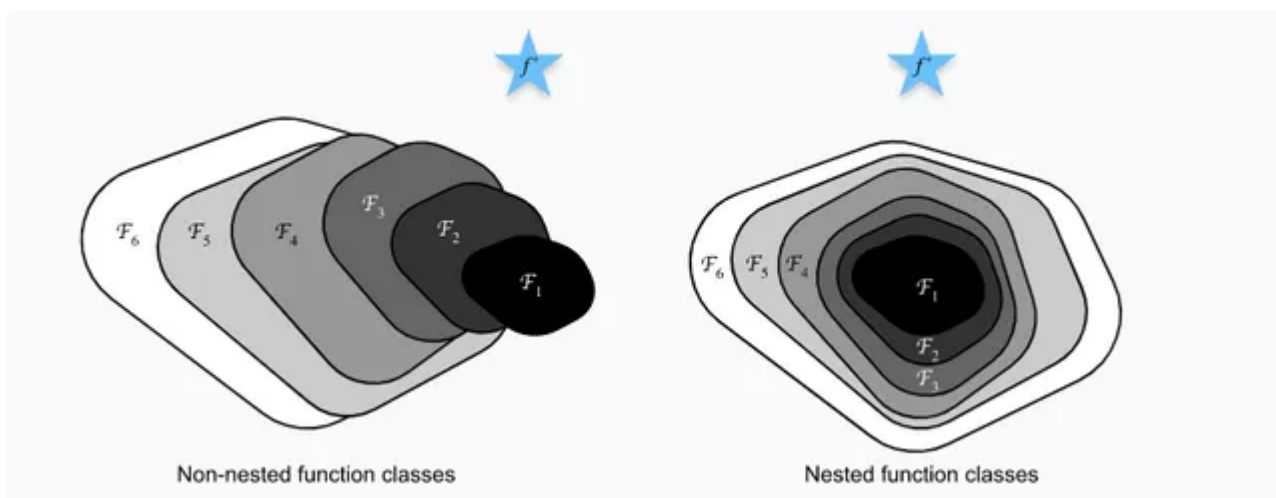
$$f_{\mathcal{F}}^* := \underset{f}{\operatorname{argmin}} L(\mathbf{X}, \mathbf{y}, f), f \in \mathcal{F}$$

🔗 FIRST INTUITION

It **sounds reasonable** to assume that, if we design a different and **more powerful architecture** $\mathcal{F}'$ we should **arrive at a better outcome**; that is, we would expect that $f_{\mathcal{F}'}^*$ is "better" than $f_{\mathcal{F}}^*$.

However, if $\mathcal{F} \not\subseteq \mathcal{F}'$ there is **no guarantee that this should happen**.

In fact, $f_{\mathcal{F}}^*$, **might even be worse**, as visually illustrated by the image below:



The figure shows us that, for **non-nested function classes**, a **larger function class** does **not always guarantee a better guess** of the "truth" function f^* . For instance, on the left, $\mathcal{F}_3$ is closer to f^* than $\mathcal{F}_1$, but $\mathcal{F}_6$ moves away and there is no guarantee that further increasing the complexity can reduce the distance from f^* . The right part of the figure shows that this *doesn't happen with nested function classes*.

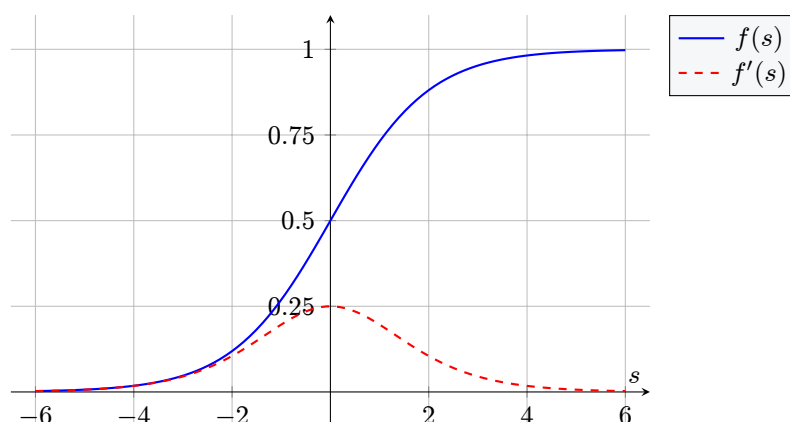
RESIDUAL NETWORK

Function classes were at the heart of the problems that He et al. were considering when working on *very deep computer vision models* for which they developed their **residual network**. See > [2.0 - CONVOLUTIONAL NEURAL NETWORKS > RESNET](#) for more details.

ACTIVATION FUNCTIONS

In MLP networks, the (historically) most used activation function is the *sigmoid*. In deep neural networks, the use of this activation function is **problematic** for the **backpropagation** algorithm because it introduces the **vanishing gradient problem** (see > [EXPLODING/VANISHING GRADIENT PROBLEM](#)).

This arises from the fact that the sigmoid has **saturation regions** at the **two ends of its domain**, where its **derivative is practically zero**:



Thus, a **large number** of different **activation functions** has been developed for deep neural networks.

NOTE

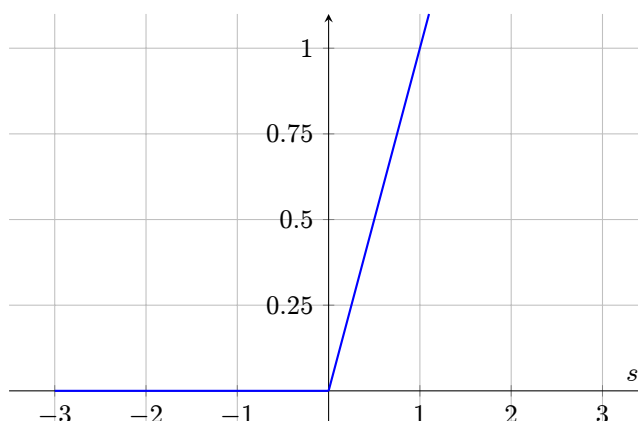
The **choice** of the **activation function** has **not yet been standardised** (officially) and is **still a widely discussed topic** in research papers. Here, we present some of the functions that appear in the literature more often, but keep in mind that **many more exist**.

ReLU

ReLU (Rectified Linear Unit)

This function is defined as:

$$f(x) = \max(0, x)$$



Notice that the function is *not differentiable* at $x = 0$, though for practical reasons its derivative is usually defined as follows:

$$f'(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

COMPARISON WITH THE SIGMOID

There is **no saturation for positive values** since the **derivative is constant for positive values**.

Furthermore, because the **derivative is null** for negative values, ReLU leads to **sparse activations** (some of the neurons are switched off), which have been observed to provide **robustness** to the network.

LEAKY ReLU, ELU

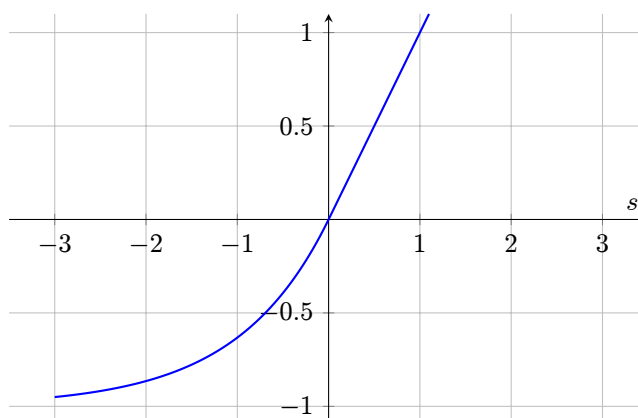
The derivative being null for all negative values **isn't always beneficial**: thus, ReLU suffers from the **dying ReLU problem**.

ELU and **Leaky ReLU** were both developed to solve this problem.

ELU (Exponential Linear Unit)

$$f(x) = \begin{cases} a(e^x - 1) & x < 0 \\ x & x \geq 0 \end{cases}$$

With $a \in \mathbb{R}$.



Thus the derivative is given by:

$$f'(x) = \begin{cases} ae^x & x < 0 \\ 1 & x \geq 0 \end{cases}$$

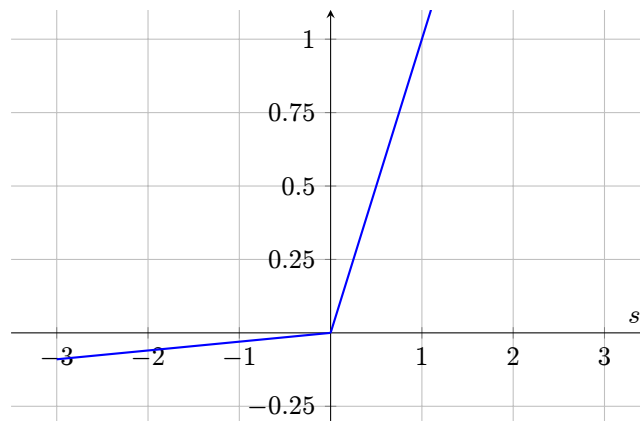
Notice that we still have $ae^x \rightarrow 0$ when $x \rightarrow -\infty$, which means **ELU weakly solves the dying ReLU problem**.

A more robust solution is provided by ReLU.

Leaky ReLU

$$f(x) = \begin{cases} ax & x < 0 \\ x & x \geq 0 \end{cases}$$

Where $a \in \mathbb{R}$ is usually **very small** (e.g. 0.01).



Thus the derivative is given by:

$$f'(x) = \begin{cases} a & x < 0 \\ 1 & x \geq 0 \end{cases}$$

PReLU

In some networks, the parameter a in Leaky ReLU can be learned by the model.

We refer to such implementations as **Parametric ReLU (PReLU)**, denoted by PReLU^a .

SReLU

S-Shaped ReLU is composed of **three piece-wise linear functions** expressed by **four learnable parameters**:

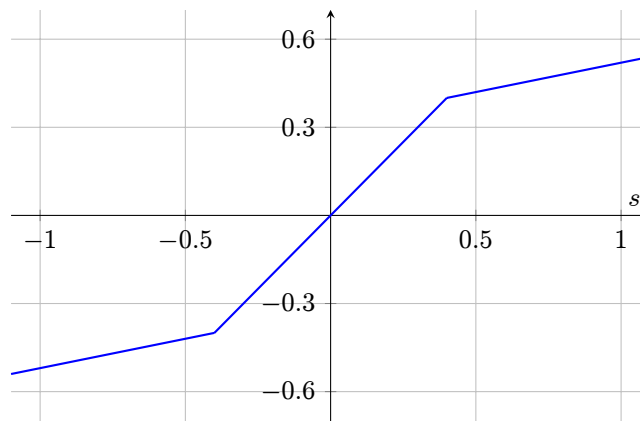
SReLU

$$f(x) = \begin{cases} t_i^r + a_i^r(x - t_i^r) & x \geq t_i^r \\ x & t_i^l < x < t_i^r \\ t_i^l + a_i^l(x - t_i^l) & x \leq t_i^l \end{cases}$$

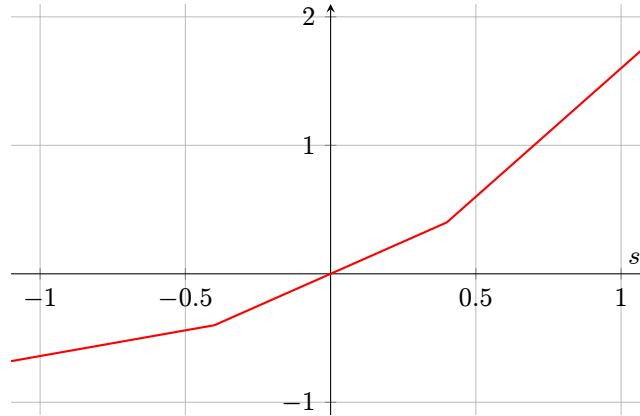
Notice the *subscript i* on the parameters: this notation indicates that there is **no fixed rule on the number of activation function parameters in the network**.

In other words, each neuron can potentially have its **own unique parameters**.

Below, we can see the resulting function for $t^r = 0.4$, $a^r = 0.2$, $t^l = -0.4$ and $a^l = 0.2$.



Below, we can see the resulting function for $t^r = 0.4$, $a^r = 2.0$, $t^l = -0.4$ and $a^l = 0.4$.



The derivative is simply:

$$f'(x) = \begin{cases} a_i^r & x_i \geq t_i^r \\ 1 & t_i^l < x < t_i^r \\ a_i^l & x \leq t_i^l \end{cases}$$

The **learnable parameters** are t^r, t^l, a^r, a^l , usually initialised as follows:

$$a^l = 0 \quad t^l = 0 \quad a^r = 1 \quad a^l = \text{maxInput}$$

Where `maxInput` is a **hyperparameter**.

Notice that the **partial derivatives** of SReLU with respect to the *learnable parameters* are:

$$\frac{\partial f}{\partial t_i^r} = I\{x \geq t_i^r\}(1 - a_i^r)$$

$$\frac{\partial f}{\partial a_i^r} = I\{x \geq t_i^r\}(x - t_i^r)$$

$$\frac{\partial f}{\partial t_i^l} = I\{x \leq t_i^l\}(1 - a_i^l)$$

$$\frac{\partial f}{\partial a_i^l} = I\{x \leq t_i^l\}(x - t_i^l)$$

These are needed for **backpropagation**. Since we added *more learnable parameters*, we also need to compute:

$$\frac{\partial J}{\partial t_i^r} \quad \frac{\partial J}{\partial a_i^r}$$

$$\frac{\partial J}{\partial t_i^l} \quad \frac{\partial J}{\partial a_i^l}$$

💡 OBSERVATION

SReLU's rather large **number of parameters** results in **high representation power**.

APLU

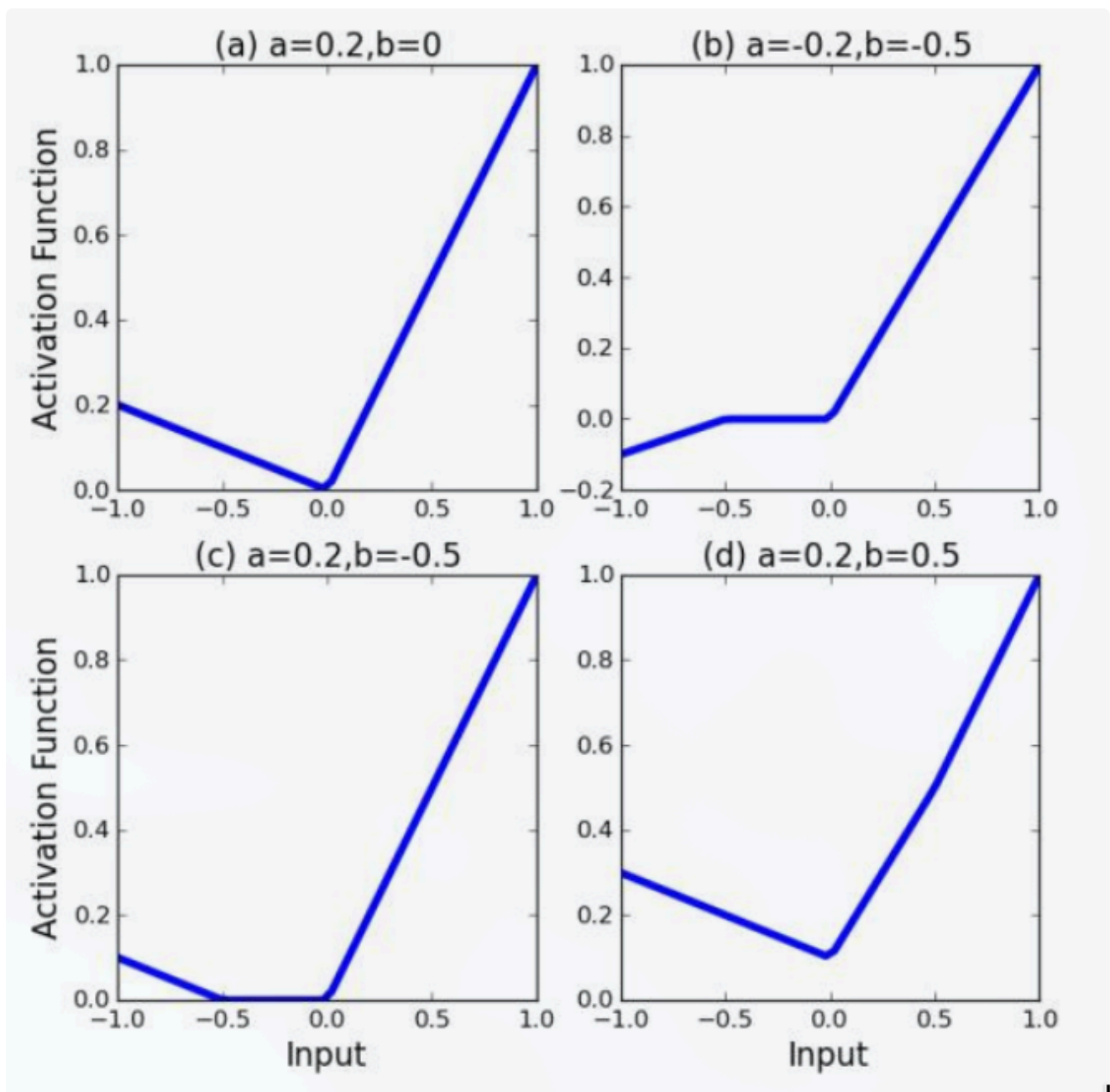
Adaptive Piecewise Linear Unit combines ReLU with a **varying number of linear pieces**.

It is defined as follows:

📖 APLU

$$f(x) = \text{ReLU}(x) + \sum_{c=1}^n a_c \max(0, b_c - x)$$

Where n is a **hyperparameter**, while the a_c coefficients are **learned during training**.



🔍 OBSERVATION

Notice that b_c controls the **position of the "hinges"**.

✍️ REGULARISATION

During training, these parameters **are accounted for by regularisation** methods, but with a **penalty**, usually set to 0.001 (this means the corresponding $J_{\text{reg,act}}$ term is multiplied by 0.001).

This is because, without the penalty, the optimiser would be *free to choose very large values of a_c and b_c balanced by very small weights*, which would lead to **numerical instability**.

MEXICAN ReLU

First, we need to introduce a "Mexican hat"-shaped function:

📖 MEXICAN HAT FUNCTION

$$\phi^{a,\lambda}(x) = \max(0, \lambda - |x - a|)$$

Where a, λ are real numbers.

These functions serve as the building blocks for **MeLU**, which is defined as follows:

📖 MEXICAN ReLU (MeLU)

$$f(x) = \text{MeLU}(x) = \text{PReLU}^{c_0}(x) + \sum_{j=1}^{k-1} c_j \phi^{a_j, \lambda_j}(x)$$

Where k is the **number of learnable parameters** for each channel (c_0 for PReLU, $k - 1$ for the Mexican hat functions: the c_j coefficients), and a_j, λ_j are **fixed and chosen recursively** starting from a `maxInput` parameter.

value of the first seven Mexican hat functions

j	1	2	3	4	5	6	7
a_j	512	256	768	128	384	640	896
λ_j	512	256	256	128	128	128	128

Fixed parameters of MeLU with *maxInput* = 256.

The figure below shows how these functions look like:

